02 — Frontend and API Integration Guide

Audience: Member 2 (Frontend Developer). **Status:** Implementation handbook. Every schema, endpoint, field name, environment variable, and screen behavior in this document must match `docs/01-MASTER-PROJECT-DEVELOPMENT-GUIDE.md` ("Document 1") exactly. If anything here ever appears to disagree with Document 1, Document 1 is correct — stop and reconcile with Member 1 (the team lead) before writing code.

---

1. Scope

Member 2 owns everything under `client/src/pages/`, `client/src/components/`, `client/src/services/`, `client/src/context/`, `client/src/styles/`, plus `client/src/App.jsx` and `client/src/main.jsx`. That is the entire frontend application. Member 2 never edits anything under `server/` — not even "just to test something" — because the backend is owned by Member 3 (core Express app, models, auth, CRUD routes) and Member 4 (retailer providers, scheduler, notifications), and an unreviewed cross-edit is exactly the kind of "works on my machine" bug the team is trying to avoid (Document 1, Section 1.3). If a page needs an API endpoint that doesn't exist yet, or needs a response shaped differently than Document 1 says, that is an **API contract change**, and API contract changes are proposed to and approved by Member 1 (the team lead), then updated in Document 1 first — never worked around by inventing a slightly different frontend assumption. Everything in this guide assumes the contract in Document 1 Section 6 (REST API), Section 5 (database field names, which show up in API responses), Section 7.2 (the one frontend environment variable), Section 9 (alert logic, which drives the alert-creation UI), and Section 13 (the full UI specification) as fixed and final.

---

2. Day 1 Windows setup

Before writing any frontend code, confirm your machine is ready. Open **PowerShell** (not Command Prompt — the whole project standardizes on PowerShell per Document 1).

2.1 Check Node.js

```
node --version
  npm --version
```

You need **Node.js v20 or higher** (the LTS version the whole team agreed on in Document 1 Section 3). If `node --version` prints anything below `v20`, or the command isn't recognized at all, install Node 20 LTS from nodejs.org (download the "LTS" installer, not "Current"), then close and reopen PowerShell and re-run the check.

2.2 Check Git

```
git --version
```

If this isn't recognized, install Git for Windows from git-scm.com. Full Git setup, identity configuration, and the complete branch/PR workflow are covered in **Document 5** — this guide only repeats the parts that are specific to frontend work (Section 15 below).

2.3 VS Code extensions (optional, recommended)

These aren't required to build the project, but they make a 7-day sprint noticeably smoother:

Extension	Why
ESLint	Flags obvious JavaScript mistakes (unused variables, undefined references) before you even run the app
Prettier - Code formatter	Auto-formats code on save so four people's code doesn't look like four different styles
ES7+ React/Redux/React-Native snippets	Faster boilerplate for functional components

None of these are a blocker — skip them if you want to start coding immediately and come back later.

2.4 Clone the repository

Once Member 1 has created the GitHub repository and added you as a collaborator (Document 5 covers this from the team-lead side), clone it:

```
cd D:\Projects
  git clone https://github.com/<org-or-owner>/<repo-name>.git PriceRadar
  cd PriceRadar
```

Full Git mechanics — configuring your identity, pulling, branching, resolving conflicts — are in **Document 5**. This guide covers only the frontend-specific branch workflow in Section 15.

---

3. Project scaffold

What is Vite, briefly? Vite is the tool that creates your React project and runs the local dev server. Older tools like Create React App bundle your *entire* app before showing you anything, which gets slower as the app grows. Vite instead serves your files almost instantly and only compiles the piece of code you're actually looking at, so the dev server starts in under a second and changes appear in the browser near-instantly. That matters a lot in a 7-day sprint where you're saving and reloading constantly.

3.1 Create the Vite project

From the repository root (`D:\Projects\PriceRadar`), run:

```
npm create vite@latest client
```

This launches an interactive prompt. Choose:

- **Framework:** React
- **Variant:** JavaScript (not TypeScript — Document 1 Section 3/4 specifies plain JavaScript, no `.tsx` files, to keep things simple for a beginner team)

This creates the `client/` folder with a working starter React app at `client/package.json`, `client/index.html`, `client/src/App.jsx`, etc. You'll reorganize and replace most of the starter content in Section 4 below.

3.2 Install dependencies

```
cd client
  npm install
```

This reads `client/package.json` and downloads every listed package into `client/node_modules/`. `node_modules/` is never committed to Git (it's in `.gitignore`) — every teammate runs `npm install` themselves after cloning.

Now add the two packages this project needs beyond the Vite starter:

```
npm install react-router-dom axios
```

- **react-router-dom** — client-side routing (so `/search`, `/dashboard`, `/alerts/new` etc. all work without a full page reload). Document 1 Section 3 specifies React Router v6.
- **axios** — the HTTP client used everywhere in this project (Document 1 Section 3: "used everywhere, consistently — not mixed with `fetch`"). Do not use the browser's built-in `fetch` anywhere in this codebase; it's one more thing for four beginners to keep straight, and Axios gives clearer error objects (Section 5 below explains why this matters).

Do not add any other npm package without checking with Member 1 first — Document 1 Section 3 explicitly calls out unplanned dependencies as a common source of "works on my machine" bugs.

3.3 Add the Bootstrap 5 CDN tags

Document 1 Section 3 specifies Bootstrap 5 **via CDN**, not an npm package — this avoids any extra build-tool configuration. Open `client/index.html` and add the Bootstrap CSS `<link>` inside `<head>` and the Bootstrap JS bundle `<script>` just before the closing `</body>` tag:

```
<!doctype html>
  <html lang="en">
    <head>
      <meta charset="UTF-8" />
      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
      <title>PriceRadar</title>



      <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet"
        integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALE
  wIH"
        crossorigin="anonymous"
      />
    </head>
    <body>
      <div id="root"></div>
      <script type="module" src="/src/main.jsx"></script>



      <script
        src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.
  js"
        integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jI
  eHz"
        crossorigin="anonymous"
      ></script>
    </body>
  </html>
```

This gives you Bootstrap's cards, badges, buttons, forms, navbar, and spinner components out of the box — no npm install, no build step. You reference Bootstrap's classes (`className="card"`, `className="badge bg-success"`, etc.) directly in your JSX.

3.4 Create your `.env` file

Document 1 Section 7.2 defines exactly **one** frontend environment variable: `VITE_API_URL`. Create `client/.env.example` (committed to Git, placeholder only) and `client/.env` (never committed — already covered by `.gitignore` from the Vite scaffold, but double-check):

`client/.env.example`:

```
VITE_API_URL=http://localhost:5000
```

`client/.env` (your own local copy — copy the example and it's already correct for local dev):

```
Copy-Item .env.example .env
```

Why the `VITE_` prefix matters: Vite only exposes environment variables to your frontend code if their name starts with `VITE_`. This is a deliberate safety net (Document 1 Section 7.2) — it means a secret like a database password or API key can *never* accidentally leak into the frontend bundle just by existing in an `.env` file, because it would need the `VITE_` prefix to be readable at all, and nothing secret is ever given that prefix. Never create a second `VITE_`-prefixed variable without checking Document 1 first.

3.5 Verify the dev server starts

From `client/`:

```
npm run dev
```

You should see output like:

```
  VITE v5.x.x  ready in 320 ms

    ➜  Local:   http://localhost:5173/
```

Open `http://localhost:5173/` in a browser — you should see the default Vite + React starter page. If you see an error instead, check Section 14 (Debugging) before asking a teammate.

Leave this running in its own PowerShell window while you work — Vite reloads the browser automatically every time you save a file.

---

4. Folder structure

This is the exact `client/` tree from Document 1 Section 4. After the scaffold step above, you'll reorganize the Vite starter's default `src/` into this shape:

```
client/
  ├── public/
  ├── src/
  │   ├── components/        # reusable UI pieces (OfferCard, AlertForm, Navbar, ...)
  │   ├── pages/              # one file per route (SearchPage.jsx, DashboardPage.jsx,
  ...)
  │   ├── services/           # api.js — the one shared Axios client
  │   ├── context/             # AuthContext.jsx
  │   ├── styles/              # theme.css
  │   ├── App.jsx
  │   └── main.jsx
  ├── index.html
  ├── package.json
  └── .env.example
```

Folder/file	Purpose
`public/`	Static files served as-is (favicon, etc.) — Vite's default, nothing special needed here for this project
`src/components/`	Small, reusable pieces used across more than one page — a card, a badge, a form, a nav bar — nothing here owns a route
`src/pages/`	One file per route/screen — `SearchPage.jsx`, `DashboardPage.jsx`, `LoginPage.jsx`, etc. — each page composes components and calls `services/api.js`
`src/services/api.js`	The single shared Axios instance and every API-call helper function — see Section 5
`src/context/AuthContext.jsx`	Holds the logged-in user and JWT in React state, shared across the whole app — see Section 6
`src/styles/theme.css`	One custom stylesheet for anything Bootstrap's defaults don't cover (the project's color palette, spacing tweaks) — see Section 12
`src/App.jsx`	The route table — maps URLs to pages — see Section 7
`src/main.jsx`	The entry point: mounts `<App />` into the DOM, wraps it in `<BrowserRouter>` and `<AuthProvider>`

Create the empty folders now (Vite's starter only creates `src/assets/` by default, which you can delete or ignore):

```
cd client/src
  New-Item -ItemType Directory -Path components, pages, services, context, styles
```

---

5. The shared API client (`client/src/services/api.js`)

5.1 Why one shared file instead of `fetch` / `axios` calls scattered across components

If every component wrote its own `axios.get(...)` call, you'd end up repeating the same base URL, the same "attach the JWT" logic, and the same error-handling pattern in a dozen places — and the day one detail changes (say, the token's localStorage key), you'd have to find and fix it everywhere. Centralizing it in one file means every page and component calls a plain function like `api.searchProducts(q)` and never thinks about headers, URLs, or tokens at all. This is a core beginner lesson in frontend architecture: **one source of truth for how the app talks to the server.**

5.2 What an Axios interceptor is

An **interceptor** is a function Axios runs automatically on every single request (or response) before it's sent (or after it comes back), without you having to call it manually each time. Here, we use a **request interceptor** to attach the `Authorization` header to every outgoing request automatically — so none of your page code ever has to remember to do it.

5.3 Complete code

Create `client/src/services/api.js`:

```
import axios from "axios";

  // The one shared Axios instance for the whole app.
  // Every request goes through this — never create a second axios instance
  // and never use the raw fetch() API anywhere in this codebase.
  const api = axios.create({
    baseURL: import.meta.env.VITE_API_URL,
  });

  // localStorage key for the JWT — must match AuthContext.jsx exactly.
  const TOKEN_KEY = "priceradar_token";

  // --- Request interceptor ------------------------------------------------
  // Runs before every request leaves the browser. If a token is saved in
  // localStorage, attach it as "Authorization: Bearer <token>". Routes that
  // don't require auth simply ignore the header on the server side.
  api.interceptors.request.use((config) => {
    const token = localStorage.getItem(TOKEN_KEY);
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  });

  // --- Response helper ----------------------------------------------------
  // Every successful backend response is shaped { success: true, data: {...} }
  // (Document 1, Section 6). This helper unwraps it so callers just get the
  // payload. Errors are shaped { success: false, error: { code, message } };
  // we let those reject normally and let each caller decide how to show them
  // (see Section 10, StatusWrapper, and Section 14, Debugging).
  function unwrap(responsePromise) {
    return responsePromise.then((res) => res.data.data);
  }

  // ============================================================================
  // Auth — Document 1 Section 6.1
  // ============================================================================

  export function register({ name, email, password }) {
    return unwrap(api.post("/api/auth/register", { name, email, password }));
  }
```

```
export function login({ email, password }) {
  return unwrap(api.post("/api/auth/login", { email, password }));
}

export function getMe() {
  return unwrap(api.get("/api/auth/me"));
}

// ============================================================================
// Products and offers — Document 1 Section 6.2
// ============================================================================

export function searchProducts(q) {
  return unwrap(api.get("/api/products/search", { params: { q } }));
}

export function getProduct(productId) {
  return unwrap(api.get(`/api/products/${productId}`));
}

export function getOfferHistory(offerId) {
  return unwrap(api.get(`/api/offers/${offerId}/history`));
}

// ============================================================================
// Watchlist — Document 1 Section 6.3
// ============================================================================

export function getWatchlist() {
  return unwrap(api.get("/api/watchlist"));
}

export function addToWatchlist(offerId) {
  return unwrap(api.post("/api/watchlist", { offerId }));
}

export function removeFromWatchlist(watchlistItemId) {
  return unwrap(api.delete(`/api/watchlist/${watchlistItemId}`));
}

// ============================================================================
// Alerts — Document 1 Section 6.4
// ============================================================================
```

```
export function getAlerts() {
  return unwrap(api.get("/api/alerts"));
}

export function createAlert({ offerId, type, targetPriceInPaise, dropAmountInPaise, c
hannels }) {
  return unwrap(
    api.post("/api/alerts", {
      offerId,
      type,
      targetPriceInPaise,
      dropAmountInPaise,
      channels,
    })
  );
}

export function updateAlert(alertId, changes) {
  // changes is any subset of { status, targetPriceInPaise, dropAmountInPaise, channe
ls }
  return unwrap(api.patch(`/api/alerts/${alertId}`, changes));
}

export function deleteAlert(alertId) {
  return unwrap(api.delete(`/api/alerts/${alertId}`));
}

// ============================================================================
// Notifications — Document 1 Section 6.5
// ============================================================================

export function getNotifications() {
  return unwrap(api.get("/api/notifications"));
}

export function getNotificationPreferences() {
  return unwrap(api.get("/api/notification-preferences"));
}

export function updateNotificationPreferences({ email }) {
  return unwrap(api.put("/api/notification-preferences", { email }));
}
```

```
export function linkTelegram(linkCode) {
  return unwrap(api.post("/api/telegram/link", { linkCode }));
}

export default api;
```

A few things worth noticing as a beginner:

- Every function returns a **Promise** — callers use `await` or `.then()`.
- `unwrap()` is in one place, so if Member 1 ever changes the envelope shape, there's exactly one line to update.
- None of these functions catch errors — that's intentional. The *page* calling them decides what "loading failed" should look like (Section 10), so errors are allowed to reject and propagate up to a `try/catch` in the component.
- `createAlert` and `updateAlert` pass `targetPriceInPaise` / `dropAmountInPaise` — not `targetPrice` or `price` — because that's the exact field name in Document 1 Section 5.6 and Section 6.4. Getting field names wrong here is the single most common frontend/backend integration bug.

---

6. AuthContext (`client/src/context/AuthContext.jsx`)

6.1 What React Context is, in beginner terms

Without Context, if `App.jsx` holds "who is logged in," and a deeply nested component six levels down needs that info, you'd have to pass it as a prop through every single component in between — even ones that don't care about it at all. This is called "prop drilling" and it gets painful fast. **React Context** lets you put a piece of state (here: the logged-in user and their token) in one place at the top of the app, and any component anywhere underneath can read it directly with a hook — no passing props through five layers of components that don't need it.

6.2 The localStorage trade-off (stated exactly as Document 1 frames it)

Document 1 Section 8 is explicit about this, and this document must not contradict it: the JWT is stored in `localStorage` under the key `priceradar_token`. **This is a deliberate, documented trade-off, not a claim that it's maximally secure.** `localStorage` is readable by any JavaScript running on the page, so it's vulnerable if the app ever has an XSS bug. The alternative — an httpOnly cookie — needs CSRF protection and cross-origin cookie configuration, which is a lot of extra surface area for four beginners in seven days. The mitigation the team has agreed to: never render unescaped user input (React escapes by default — never use `dangerouslySetInnerHTML` anywhere in this codebase) and keep the token's lifetime short (`JWT_EXPIRES_IN=7d` on the backend is already a compromise — don't ask Member 3 to extend it).

6.3 Complete code

Create `client/src/context/AuthContext.jsx`:

```
import { createContext, useContext, useState, useCallback } from "react";
  import * as api from "../services/api";

  const TOKEN_KEY = "priceradar_token";
  const USER_KEY = "priceradar_user";

  const AuthContext = createContext(null);

  export function AuthProvider({ children }) {
    // Initialize from localStorage so a page refresh doesn't log the user out.
    const [token, setToken] = useState(() => localStorage.getItem(TOKEN_KEY));
    const [user, setUser] = useState(() => {
      const stored = localStorage.getItem(USER_KEY);
      return stored ? JSON.parse(stored) : null;
    });

    const persist = (nextToken, nextUser) => {
      if (nextToken) {
        localStorage.setItem(TOKEN_KEY, nextToken);
      } else {
        localStorage.removeItem(TOKEN_KEY);
      }
      if (nextUser) {
        localStorage.setItem(USER_KEY, JSON.stringify(nextUser));
      } else {
        localStorage.removeItem(USER_KEY);
      }
      setToken(nextToken);
      setUser(nextUser);
    };

    const login = useCallback(async ({ email, password }) => {
      // api.login returns { token, user: { id, name, email } } — Document 1 Section 6.
  1
      const data = await api.login({ email, password });
      persist(data.token, data.user);
      return data.user;
    }, []);

    const register = useCallback(async ({ name, email, password }) => {
      // api.register returns { user: { id, name, email } } — it does NOT log the
      // user in automatically (no token in the response), so the Registration
```

```
    // page calls login() itself right after a successful register — see the
    // worked task for FE-05 in Section 9.
    const data = await api.register({ name, email, password });
    return data.user;
  }, []);

  const logout = useCallback(() => {
    persist(null, null);
  }, []);

  const value = {
    user,
    token,
    isAuthenticated: Boolean(token),
    login,
    register,
    logout,
  };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) {
    throw new Error("useAuth() must be called inside an <AuthProvider>");
  }
  return ctx;
}
```

Wrap the app with it in `client/src/main.jsx` (full file shown in Section 7.3 once routing is added):

```
import { AuthProvider } from "./context/AuthContext";
```

Any component can now do:

```
import { useAuth } from "../context/AuthContext";

function SomeComponent() {
  const { user, isAuthenticated, logout } = useAuth();
  // ...
}
```

— without a single prop having been passed down to get there.

---

7. Routing (`client/src/App.jsx`)

7.1 Route table

Every route below maps directly to a screen in Document 1 Section 13.

Path	Page component	Auth required?
`/`	`HomePage`	No
`/search`	`SearchPage`	No
`/products/:productId`	`ProductDetailPage`	No
`/register`	`RegisterPage`	No
`/login`	`LoginPage`	No
`/dashboard`	`DashboardPage`	**Yes**
`/watchlist`	`WatchlistPage`	**Yes**
`/alerts/new`	`AlertFormPage`	**Yes**
`/alerts/:alertId/edit`	`AlertFormPage`	**Yes**
`/alerts`	`AlertManagementPage`	**Yes**
`/settings/notifications`	`NotificationPreferencesPage`	**Yes**
`/notifications`	`NotificationHistoryPage`	**Yes**
`*` (anything else)	`NotFoundPage`	No

7.2 `<ProtectedRoute>`

Document 1 Section 13.13 specifies: on unauthorized access to a protected page, redirect to `/login` with a "Please log in to continue" message, and return to the original destination after login succeeds. Create `client/src/components/ProtectedRoute.jsx`:

```
import { Navigate, useLocation } from "react-router-dom";
  import { useAuth } from "../context/AuthContext";

  export default function ProtectedRoute({ children }) {
    const { isAuthenticated } = useAuth();
    const location = useLocation();

    if (!isAuthenticated) {
      // Carry the page the user was trying to reach in route state, so the
      // Login page can send them back after a successful login.
      return (
        <Navigate
          to="/login"
          state={{ from: location, message: "Please log in to continue." }}
          replace
        />
      );
    }

    return children;
  }
```

7.3 Complete `App.jsx` and `main.jsx`

`client/src/App.jsx`:

```
import { Routes, Route } from "react-router-dom";
  import Navbar from "./components/Navbar";
  import ProtectedRoute from "./components/ProtectedRoute";

  import HomePage from "./pages/HomePage";
  import SearchPage from "./pages/SearchPage";
  import ProductDetailPage from "./pages/ProductDetailPage";
  import RegisterPage from "./pages/RegisterPage";
  import LoginPage from "./pages/LoginPage";
  import DashboardPage from "./pages/DashboardPage";
  import WatchlistPage from "./pages/WatchlistPage";
  import AlertFormPage from "./pages/AlertFormPage";
  import AlertManagementPage from "./pages/AlertManagementPage";
  import NotificationPreferencesPage from "./pages/NotificationPreferencesPage";
  import NotificationHistoryPage from "./pages/NotificationHistoryPage";
  import NotFoundPage from "./pages/NotFoundPage";

  export default function App() {
    return (
      <>
        <Navbar />
        <main className="container py-4">
          <Routes>
            <Route path="/" element={<HomePage />} />
            <Route path="/search" element={<SearchPage />} />
            <Route path="/products/:productId" element={<ProductDetailPage />} />
            <Route path="/register" element={<RegisterPage />} />
            <Route path="/login" element={<LoginPage />} />

            <Route
              path="/dashboard"
              element={
                <ProtectedRoute>
                  <DashboardPage />
                </ProtectedRoute>
              }
            />
            <Route
              path="/watchlist"
              element={
                <ProtectedRoute>
                  <WatchlistPage />
```

```
            </ProtectedRoute>
          }
        />
        <Route
          path="/alerts/new"
          element={
            <ProtectedRoute>
              <AlertFormPage />
            </ProtectedRoute>
          }
        />
        <Route
          path="/alerts/:alertId/edit"
          element={
            <ProtectedRoute>
              <AlertFormPage />
            </ProtectedRoute>
          }
        />
        <Route
          path="/alerts"
          element={
            <ProtectedRoute>
              <AlertManagementPage />
            </ProtectedRoute>
          }
        />
        <Route
          path="/settings/notifications"
          element={
            <ProtectedRoute>
              <NotificationPreferencesPage />
            </ProtectedRoute>
          }
        />
        <Route
          path="/notifications"
          element={
            <ProtectedRoute>
              <NotificationHistoryPage />
            </ProtectedRoute>
          }
        />
```

```
            <Route path="*" element={<NotFoundPage />} />
          </Routes>
        </main>
      </>
    );
  }
```

`client/src/main.jsx`:

```
import { StrictMode } from "react";
  import { createRoot } from "react-dom/client";
  import { BrowserRouter } from "react-router-dom";
  import { AuthProvider } from "./context/AuthContext";
  import App from "./App";
  import "./styles/theme.css";

  createRoot(document.getElementById("root")).render(
    <StrictMode>
      <BrowserRouter>
        <AuthProvider>
          <App />
        </AuthProvider>
      </BrowserRouter>
    </StrictMode>
  );
```

Note the provider order: `BrowserRouter` wraps `AuthProvider` wraps `App`, because `ProtectedRoute` needs both the router (`useLocation`) and the auth context (`useAuth`) to be available.

---

8. Worked example: Search + Results (Document 1 Section 13.2–13.3)

This section gives complete, runnable code for one full feature end to end, so you can see the pattern before repeating it for the remaining pages in Section 9.

8.1 `client/src/pages/SearchPage.jsx`

Per Document 1 Section 13.2, the search bar and results live on the same route (`/search?q=...`) rather than two separate hops, with loading/empty/error states.

```
import { useState, useEffect, useCallback } from "react";
  import { useSearchParams, useNavigate } from "react-router-dom";
  import * as api from "../services/api";
  import ProductCard from "../components/ProductCard";

  export default function SearchPage() {
    const [searchParams, setSearchParams] = useSearchParams();
    const navigate = useNavigate();
    const initialQuery = searchParams.get("q") || "";

    const [inputValue, setInputValue] = useState(initialQuery);
    const [products, setProducts] = useState([]);
    const [status, setStatus] = useState("idle"); // idle | loading | success | empty |
  error
    const [lastQuery, setLastQuery] = useState(initialQuery);

    const runSearch = useCallback(async (q) => {
      if (!q.trim()) return;
      setStatus("loading");
      setLastQuery(q);
      try {
        const data = await api.searchProducts(q);
        setProducts(data.products);
        setStatus(data.products.length === 0 ? "empty" : "success");
      } catch (err) {
        setStatus("error");
      }
    }, []);

    // Run the search whenever the URL's ?q= changes (covers back/forward nav too).
    useEffect(() => {
      if (initialQuery) {
        runSearch(initialQuery);
      }
      // eslint-disable-next-line react-hooks/exhaustive-deps
    }, [initialQuery]);

    function handleSubmit(e) {
      e.preventDefault();
      setSearchParams({ q: inputValue });
    }
```

```
  return (
    <div>
      <form onSubmit={handleSubmit} className="mb-4">
        <label htmlFor="search-input" className="visually-hidden">
          Search for a product, brand, or model
        </label>
        <div className="input-group">
          <input
            id="search-input"
            type="text"
            className="form-control"
            placeholder="Search for a product, brand, or model"
            value={inputValue}
            onChange={(e) => setInputValue(e.target.value)}
          />
          <button type="submit" className="btn btn-primary">
            Search
          </button>
        </div>
      </form>

      {status === "loading" && (
        <div className="text-center py-4">
          <div className="spinner-border text-primary" role="status">
            <span className="visually-hidden">Searching…</span>
          </div>
        </div>
      )}

      {status === "empty" && (
        <p className="text-muted">
          No products matched &lsquo;{lastQuery}&rsquo;. Try a different search.
        </p>
      )}

      {status === "error" && (
        <div className="alert alert-danger d-flex justify-content-between align-items
-center">
          <span>Something went wrong searching — try again.</span>
          <button className="btn btn-outline-danger btn-sm" onClick={() => runSearch
(lastQuery)}>
            Retry
          </button>
```

```
        </div>
      )}

      {status === "success" && (
        <ul className="list-unstyled d-flex flex-column gap-3" aria-label="Search res
ults">
          {products.map((product) => (
            <li key={product.id}>
              <ProductCard product={product} onOpen={() => navigate(`/products/${prod
uct.id}`)} />
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

8.2 `client/src/components/ProductCard.jsx`

Per Document 1 Section 13.3: title, brand/variant line, up to 2 offer rows (provider, large bold price, "Sample data" badge when `isSample`, "View on `<provider>`" link opening in a new tab), the cheapest offer text-labeled "Lowest price" (never color alone — this is ACC-02 in Document 1 Section 17), and clicking the card body (not the external link) navigates to the detail page.

```
import OfferRow from "./OfferRow";

  export default function ProductCard({ product, onOpen }) {
    const { title, brand, variant, offers } = product;

    // Determine the lowest-priced offer among the ones shown on the card.
    // Document 1 Section 13.4 defines "valid" as in_stock or unknown, not error —
    // the card shows at most 2 offers so we apply the same rule here.
    const visibleOffers = offers.slice(0, 2);
    const validOffers = visibleOffers.filter((o) => o.availability !== "out_of_stock");
    const lowestOfferId =
      validOffers.length > 0
        ? validOffers.reduce((min, o) => (o.priceInPaise < min.priceInPaise ? o : mi
  n)).id
        : null;

    return (
      <div className="card shadow-sm" style={{ cursor: "pointer" }}>
        <div className="card-body" onClick={onOpen}>
          <h3 className="h6 mb-1">{title}</h3>
          {(brand || variant) && (
            <p className="text-muted small mb-3">
              {[brand, variant].filter(Boolean).join(" · ")}
            </p>
          )}

          <div className="d-flex flex-column gap-2">
            {visibleOffers.map((offer) => (
              <OfferRow key={offer.id} offer={offer} isLowest={offer.id === lowestOffer
  Id} />
            ))}
          </div>
        </div>
      </div>
    );
  }
```

8.3 `client/src/components/OfferRow.jsx`

```
export default function OfferRow({ offer, isLowest }) {
    const priceInRupees = (offer.priceInPaise / 100).toLocaleString("en-IN", {
      style: "currency",
      currency: "INR",
      maximumFractionDigits: 0,
    });

    return (
      <div className="d-flex justify-content-between align-items-center border rounded
  p-2">
        <div>
          <div className="text-capitalize fw-semibold small">{offer.provider}</div>
          <div className="fs-5 fw-bold">
            {priceInRupees}
            {isLowest && <span className="badge text-bg-success ms-2">Lowest price</spa
  n>}
          </div>
          {offer.isSample && <span className="badge bg-secondary-subtle text-secondar
  y">Sample data</span>}
        </div>
        <a
          href={offer.productUrl}
          target="_blank"
          rel="noopener noreferrer"
          className="btn btn-outline-primary btn-sm"
          onClick={(e) => e.stopPropagation()}
        >
          View on {offer.provider}
        </a>
      </div>
    );
  }
```

Notice `e.stopPropagation()` on the link's click handler — without it, clicking "View on Flipkart" would also trigger the card's `onOpen` navigation, which is exactly the bug Document 1 Section 13.3 is guarding against ("clicking the card, not the external link, goes to the comparison page").

---

9. Remaining pages — task list (FE-01 through FE-11)

The tasks below use the full 16-part template from the project brief (MASTER-PROMPT-SOURCE.md, Section 13) for every substantial feature. Scaffold/setup steps already covered concretely in Sections 2–3 are not re-wrapped in this template, per the brief's own allowance not to force administrative steps into it.

FE-01 — Project scaffold, dependencies, and environment

A. Task ID and title: FE-01 — Vite project scaffold and environment setup. **B. Owner and reviewer:** Owner: Member 2. Reviewer: Member 1. No dependencies — this is the first frontend task. **C. Purpose:** Establish the working React + Vite project every later frontend task builds on. **D. Prerequisites:** Node.js v20+, Git, repository cloned (Section 2). **E. Exact files:** Creates `client/` entirely (via `npm create vite@latest`), `client/.env.example`, `client/.env`. **F. Commands:** See Section 3 in full — `npm create vite@latest client`, `npm install`, `npm install react-router-dom axios`. **G. Implementation:** Follow Section 3.1–3.5 exactly. **H. Beginner explanation:** See Section 3's Vite explanation. **I. Environment configuration:** `VITE_API_URL=http://localhost:5000` in `client/.env`. **J. Testing:** `npm run dev` from `client/` starts a dev server on `http://localhost:5173` with no console errors. **K. Expected results:** Browser shows the default (soon to be replaced) Vite + React page. **L. Troubleshooting:** See Section 14. **M. Acceptance criteria:** `npm run dev` runs clean; `client/.env` exists and is git-ignored; `client/.env.example` is committed. **N. Git workflow:** Branch `feature/project-scaffold` — see Section 15 for the full command sequence. **O. Completion checklist:** [ ] Node 20+ confirmed [ ] `client/` created [ ] both npm packages installed [ ] Bootstrap CDN tags added to `index.html` [ ] `.env` / `.env.example` created [ ] dev server runs. **P. AI-assisted debugging prompt:** "Here is my `client/package.json` and the exact error from `npm run dev`: [paste]. Identify the root cause, make the smallest safe change to fix it, and tell me how to verify the fix."

FE-02 — API client and AuthContext

A. Task ID and title: FE-02 — Shared Axios client and authentication context. **B. Owner and reviewer:** Owner: Member 2. Reviewer: Member 1 (auth-sensitive code, per Document 1 Section 14's review rule). Depends on FE-01 and on Member 3 having `/api/auth/*` routes reachable for integration testing (can be built in parallel against the documented contract beforehand). **C. Purpose:** Give every page one consistent way to call the backend and one consistent way to know who's logged in. **D. Prerequisites:** FE-01 complete. **E. Exact files:** Create `client/src/services/api.js`, `client/src/context/AuthContext.jsx`. Do not touch any file under `server/`. **F. Commands:** None beyond what FE-01 already installed. **G. Implementation:** Use the complete code in Section 5.3 (`api.js`) and Section 6.3 (`AuthContext.jsx`) verbatim, then wire `AuthProvider` into `main.jsx` as shown in Section 7.3. **H. Beginner explanation:** See Section 5.1–5.2 (interceptors, centralization) and Section 6.1–6.2 (Context, localStorage trade-off). **I. Environment configuration:** Relies on `VITE_API_URL` from FE-01; no new variables. **J. Testing:** With the backend running locally (`npm run dev` in `server/`), open the browser console and run a manual smoke test: temporarily call `api.getMe()` from a component and confirm it throws a 401 when no token is set, then confirm `api.login(...)` with valid test credentials returns `{token, user}`. **K. Expected results:** A failed `getMe()` call rejects with a 401 response; a successful `login()` call returns an object with `token` and `user.id/name/email`. **L. Troubleshooting:** See Section 14 ("401s because the token wasn't attached", "CORS errors"). **M.**

Acceptance criteria: `localStorage` shows `priceradar_token` and `priceradar_user` after a successful `login()`; `useAuth()` is callable from any component wrapped in `<AuthProvider>` without throwing. **N. Git workflow:** Same branch as FE-01 or a new `feature/user-authentication` branch — see Section 15. **O. Completion checklist:** [ ] `api.js` exports every function listed in Section 5.3 [ ] interceptor confirmed attaching the header [ ] `AuthContext` persists across a page refresh. **P. AI-assisted debugging prompt:** "My AuthContext login() call succeeds per the Network tab but useAuth() still shows isAuthenticated: false. Here's AuthContext.jsx and the component calling login(): [paste]. Find the smallest fix, preserving the rest of the file."

FE-03 — Routing and app shell

A. Task ID and title: FE-03 — React Router route table, Navbar, ProtectedRoute. **B. Owner and reviewer:** Owner: Member 2. Reviewer: Member 1. Depends on FE-02 (needs `useAuth()`). **C. Purpose:** Make every screen in Document 1 Section 13 reachable at its specified URL, and gate the ones that require login. **D. Prerequisites:** FE-02 complete. **E. Exact files:** Create/replace `client/src/App.jsx`, `client/src/main.jsx`, `client/src/components/ProtectedRoute.jsx`, `client/src/components/Navbar.jsx`. Create empty placeholder files for every page listed in Section 7.1 so routing compiles (filled in by FE-04 through FE-11). **F. Commands:** None new. **G. Implementation:** Use the complete code in Section 7.2–7.3. For `Navbar.jsx`, implement a Bootstrap navbar per Document 1 Section 13.1 ("Nav: logo/home, Search, Login/Register (logged out) or Dashboard/Logout (logged in)"):

```
import { Link, useNavigate } from "react-router-dom";
  import { useAuth } from "../context/AuthContext";

  export default function Navbar() {
    const { isAuthenticated, logout } = useAuth();
    const navigate = useNavigate();

    function handleLogout() {
      logout();
      navigate("/");
    }

    return (
      <nav className="navbar navbar-expand navbar-light bg-white border-bottom">
        <div className="container">
          <Link className="navbar-brand fw-bold" to="/" style={{ color: "#0F6E56" }}>
            PriceRadar
          </Link>
          <div className="d-flex gap-3 align-items-center">
            <Link to="/search" className="nav-link d-inline">Search</Link>
            {isAuthenticated ? (
              <>
                <Link to="/dashboard" className="nav-link d-inline">Dashboard</Link>
                <button className="btn btn-outline-secondary btn-sm" onClick={handleLog
  out}>
                  Logout
                </button>
              </>
            ) : (
              <>
                <Link to="/login" className="nav-link d-inline">Login</Link>
                <Link to="/register" className="btn btn-primary btn-sm">Register</Link>
              </>
            )}
          </div>
        </div>
      </nav>
    );
  }
```

H. Beginner explanation: `<Routes>` / `<Route>` match the current URL to a component. `<ProtectedRoute>` wraps a page and checks `isAuthenticated` before rendering it, redirecting to `/login` otherwise and remembering where the user was headed via route `state`. **I. Environment configuration:** None new. **J. Testing:** Manually visit every path in the Section 7.1 table while logged out — protected ones should redirect to `/login`; log in and confirm they then render; visit a nonsense path and confirm `NotFoundPage` renders. **K. Expected results:** No blank screens on any route; protected routes redirect correctly. **L. Troubleshooting:** "A blank page from a router typo" — see Section 14. **M. Acceptance criteria:** Every path in the Section 7.1 table renders something (even a placeholder) with no console errors; protected routes enforce login. **N. Git workflow:** `feature/user-authentication` or a dedicated routing commit on the same branch — see Section 15. **O. Completion checklist:** [ ] All 12 routes registered [ ] ProtectedRoute redirects correctly and returns the user afterward [ ] Navbar shows the right links logged in vs out. **P. AI-assisted debugging prompt:** "Visiting /dashboard while logged out shows a blank page instead of redirecting to /login. Here's App.jsx and ProtectedRoute.jsx: [paste]. Find the smallest fix."

FE-04 — Search and results (worked example — see Section 8)

Already fully implemented in Section 8 above (`SearchPage.jsx`, `ProductCard.jsx`, `OfferRow.jsx`). Branch: `feature/product-search`. Acceptance criteria: ACC-01 and ACC-02 from Document 1 Section 17 pass against the real local backend (searching a known fixture product returns ≥2 labeled offers; the cheapest is text-labeled "Lowest price").

FE-05 — Registration and Login pages

A. Task ID and title: FE-05 — Registration and Login pages. **B. Owner and reviewer:** Owner: Member 2. Reviewer: Member 1. Depends on FE-02, FE-03. **C. Purpose:** Let a new user create an account and an existing user sign in, per Document 1 Sections 13.5–13.6. **D. Prerequisites:** Backend `/api/auth/register` and `/api/auth/login` reachable (Member 3's work; can stub against the documented contract while waiting). **E. Exact files:** Create `client/src/pages/RegisterPage.jsx`, `client/src/pages/LoginPage.jsx`, `client/src/components/FormField.jsx` (small reusable labeled-input-plus-error component). **F. Commands:** None new. **G. Implementation:**

`client/src/components/FormField.jsx`:

```
export default function FormField({ id, label, type = "text", value, onChange, error,
  ...rest }) {
    return (
      <div className="mb-3">
        <label htmlFor={id} className="form-label">{label}</label>
        <input
          id={id}
          type={type}
          className={`form-control${error ? " is-invalid" : ""}`}
          value={value}
          onChange={onChange}
          {...rest}
        />
        {error && <div className="invalid-feedback">{error}</div>}
      </div>
    );
  }
```

`client/src/pages/RegisterPage.jsx` (validation per Document 1 Section 13.5: all required, email format, password ≥ 8 chars, confirm must match, submit disabled while invalid, server `409 CONFLICT` shown under Email):

```
import { useState } from "react";
  import { useNavigate } from "react-router-dom";
  import { useAuth } from "../context/AuthContext";
  import FormField from "../components/FormField";

  const EMAIL_RE = /^[^\s@]+@[^\s@]+.[^\s@]+$/;

  export default function RegisterPage() {
    const { register, login } = useAuth();
    const navigate = useNavigate();

    const [form, setForm] = useState({ name: "", email: "", password: "", confirm: ""
  });
    const [errors, setErrors] = useState({});
    const [submitting, setSubmitting] = useState(false);
    const [serverError, setServerError] = useState("");

    function validate() {
      const e = {};
      if (!form.name.trim()) e.name = "Name is required.";
      if (!form.email.trim()) e.email = "Email is required.";
      else if (!EMAIL_RE.test(form.email)) e.email = "Enter a valid email address.";
      if (!form.password) e.password = "Password is required.";
      else if (form.password.length < 8) e.password = "Password must be at least 8 char
  acters.";
      if (form.confirm !== form.password) e.confirm = "Passwords do not match.";
      return e;
    }

    const liveErrors = validate();
    const isValid = Object.keys(liveErrors).length === 0;

    function handleChange(field) {
      return (e) => setForm((f) => ({ ...f, [field]: e.target.value }));
    }

    async function handleSubmit(e) {
      e.preventDefault();
      const v = validate();
      setErrors(v);
      if (Object.keys(v).length > 0) return;
```

```
    setSubmitting(true);
    setServerError("");
    try {
      await register({ name: form.name, email: form.email, password: form.password
});
      // Registration succeeds without a token (Document 1 Section 6.1) —
      // log the user in immediately so they land on the Dashboard signed in.
      await login({ email: form.email, password: form.password });
      navigate("/dashboard");
    } catch (err) {
      const apiError = err?.response?.data?.error;
      if (apiError?.code === "CONFLICT") {
        setErrors((prev) => ({ ...prev, email: apiError.message }));
      } else {
        setServerError(apiError?.message || "Something went wrong — please try agai
n.");
      }
    } finally {
      setSubmitting(false);
    }
  }

  return (
    <div className="mx-auto" style={{ maxWidth: 420 }}>
      <h1 className="h4 mb-3">Create your account</h1>
      {serverError && <div className="alert alert-danger">{serverError}</div>}
      <form onSubmit={handleSubmit} noValidate>
        <FormField id="name" label="Name" value={form.name} onChange={handleChange("n
ame")} error={errors.name} />
        <FormField id="email" label="Email" type="email" value={form.email} onChange=
{handleChange("email")} error={errors.email} />
        <FormField id="password" label="Password" type="password" value={form.passwor
d} onChange={handleChange("password")} error={errors.password} />
        <FormField id="confirm" label="Confirm password" type="password" value={form.
confirm} onChange={handleChange("confirm")} error={errors.confirm} />
        <button type="submit" className="btn btn-primary w-100" disabled={!isValid ||
submitting}>
          {submitting ? (
            <span className="spinner-border spinner-border-sm me-2" />
          ) : null}
          Create account
        </button>
      </form>
```

```
      </div>
    );
  }
```

`client/src/pages/LoginPage.jsx` (redirect-to-original-destination per Section 13.13, `401 AUTH_INVALID` → "Email or password is incorrect"):

```
import { useState } from "react";
  import { useNavigate, useLocation, Link } from "react-router-dom";
  import { useAuth } from "../context/AuthContext";
  import FormField from "../components/FormField";

  export default function LoginPage() {
    const { login } = useAuth();
    const navigate = useNavigate();
    const location = useLocation();

    const [form, setForm] = useState({ email: "", password: "" });
    const [errors, setErrors] = useState({});
    const [submitting, setSubmitting] = useState(false);
    const [serverError, setServerError] = useState("");

    const redirectTo = location.state?.from?.pathname || "/dashboard";
    const infoMessage = location.state?.message;

    const isValid = form.email.trim() !== "" && form.password !== "";

    function handleChange(field) {
      return (e) => setForm((f) => ({ ...f, [field]: e.target.value }));
    }

    async function handleSubmit(e) {
      e.preventDefault();
      if (!isValid) return;
      setSubmitting(true);
      setServerError("");
      setErrors({});
      try {
        await login(form);
        navigate(redirectTo, { replace: true });
      } catch (err) {
        const apiError = err?.response?.data?.error;
        if (apiError?.code === "AUTH_INVALID") {
          setServerError("Email or password is incorrect.");
        } else {
          setServerError(apiError?.message || "Something went wrong — please try agai
  n.");
        }
      } finally {
```

```
      setSubmitting(false);
    }
  }

  return (
    <div className="mx-auto" style={{ maxWidth: 420 }}>
      <h1 className="h4 mb-3">Log in</h1>
      {infoMessage && <div className="alert alert-info">{infoMessage}</div>}
      {serverError && <div className="alert alert-danger">{serverError}</div>}
      <form onSubmit={handleSubmit} noValidate>
        <FormField id="email" label="Email" type="email" value={form.email} onChange=
{handleChange("email")} error={errors.email} />
        <FormField id="password" label="Password" type="password" value={form.passwor
d} onChange={handleChange("password")} error={errors.password} />
        <button type="submit" className="btn btn-primary w-100" disabled={!isValid ||
submitting}>
          {submitting ? <span className="spinner-border spinner-border-sm me-2" /> :
null}
          Log in
        </button>
      </form>
      <p className="text-center mt-3 small">
        No account? <Link to="/register">Register</Link>
      </p>
    </div>
  );
}
```

H. Beginner explanation: Validation runs on every keystroke (`validate()` is recomputed each render) so the submit button disables itself the instant the form becomes invalid, without waiting for a submit attempt. The server error distinction (`CONFLICT` → inline under Email, anything else → a banner) matches exactly what Document 1 Section 13.5 specifies. **I. Environment configuration:** None new. **J. Testing:** Register a new account → should log in automatically and land on `/dashboard`. Register the same email again → should show "already registered" (or whatever message the backend sends) under the Email field, not a generic banner. Try logging in with a wrong password → "Email or password is incorrect." **K. Expected results:** Successful register+login results in `priceradar_token` in localStorage and a `/dashboard` redirect. **L. Troubleshooting:** 401s / token-not-attached — Section 14. **M. Acceptance criteria:** All validation rules from Document 1 Section 13.5–13.6 enforced client-side before any request is sent; server errors surfaced in the right place. **N. Git workflow:** `feature/user-authentication` — see Section 15. **O. Completion checklist:** [ ] Both forms validate client-side [ ] submit disabled while invalid/pending [ ] CONFLICT and AUTH_INVALID handled distinctly [ ]

redirect-after-login works. **P. AI-assisted debugging prompt:** "After a successful login, the app doesn't redirect to the page I came from — it always goes to /dashboard. Here's LoginPage.jsx and ProtectedRoute.jsx: [paste]. Make the smallest fix."

FE-06 — Product comparison / detail page

A. Task ID and title: FE-06 — Product detail/comparison page (`/products/:productId`). **B. Owner and reviewer:** Owner: Member 2. Reviewer: Member 1. Depends on FE-04 (card click navigates here), Member 3's `GET /api/products/:productId` route. **C. Purpose:** Full price comparison across every offer for one product, per Document 1 Section 13.4. **D. Prerequisites:** FE-02, FE-03. **E. Exact files:** Create `client/src/pages/ProductDetailPage.jsx`. **F. Commands:** None new. **G. Implementation:**

```
import { useState, useEffect, useCallback } from "react";
  import { useParams } from "react-router-dom";
  import * as api from "../services/api";
  import { useAuth } from "../context/AuthContext";

  function formatLastUpdated(offer) {
    if (offer.lastStatus === "error") return "Error fetching latest price";
    if (!offer.lastSuccessAt) return "Stale — not yet checked";
    const minutesAgo = Math.round((Date.now() - new Date(offer.lastSuccessAt).getTime
  ()) / 60000);
    if (offer.lastStatus === "stale") {
      const daysAgo = Math.max(1, Math.round(minutesAgo / 1440));
      return `Stale — last checked ${daysAgo} day${daysAgo === 1 ? "" : "s"} ago`;
    }
    if (minutesAgo < 1) return "Updated just now";
    if (minutesAgo < 60) return `Updated ${minutesAgo} minute${minutesAgo === 1 ? "" :
  "s"} ago`;
    const hoursAgo = Math.round(minutesAgo / 60);
    return `Updated ${hoursAgo} hour${hoursAgo === 1 ? "" : "s"} ago`;
  }

  export default function ProductDetailPage() {
    const { productId } = useParams();
    const { isAuthenticated } = useAuth();

    const [data, setData] = useState(null);
    const [status, setStatus] = useState("loading"); // loading | success | empty | err
  or
    const [actionMessage, setActionMessage] = useState("");

    const load = useCallback(async () => {
      setStatus("loading");
      try {
        const result = await api.getProduct(productId);
        if (!result.product) {
          setStatus("empty");
          return;
        }
        setData(result);
        setStatus("success");
      } catch (err) {
        setStatus("error");
```

```
    }
  }, [productId]);

  useEffect(() => {
    load();
  }, [load]);

  async function handleAddToWatchlist(offerId) {
    if (!isAuthenticated) {
      window.location.href = `/login?returnTo=/products/${productId}`;
      return;
    }
    try {
      await api.addToWatchlist(offerId);
      setActionMessage("Added to your watchlist.");
    } catch (err) {
      setActionMessage("Could not add to watchlist — try again.");
    }
  }

  if (status === "loading") {
    return (
      <div className="d-flex flex-column gap-2">
        {[1, 2, 3].map((i) => (
          <div key={i} className="placeholder-glow">
            <span className="placeholder col-12" style={{ height: 60, display: "bloc
k" }} />
          </div>
        ))}
      </div>
    );
  }

  if (status === "empty") {
    return <p className="text-muted">This product isn&apos;t tracked yet.</p>;
  }

  if (status === "error") {
    return (
      <div className="alert alert-danger d-flex justify-content-between align-items-c
enter">
        <span>Something went wrong loading this product.</span>
        <button className="btn btn-outline-danger btn-sm" onClick={load}>Retry</butto
```

```
n>
      </div>
    );
  }

  const { product, offers } = data;
  const sortedOffers = [...offers].sort((a, b) => a.priceInPaise - b.priceInPaise);
  const firstValidIndex = sortedOffers.findIndex((o) => o.lastStatus !== "error");

  return (
    <div>
      <h1 className="h4">{product.title}</h1>
      <p className="text-muted">{[product.brand, product.variant].filter(Boolean).joi
n(" · ")}</p>

      {actionMessage && <div className="alert alert-info">{actionMessage}</div>}

      <div className="d-flex flex-column gap-2 mt-3">
        {sortedOffers.map((offer, index) => (
          <div key={offer.id} className="card">
            <div className="card-body d-flex justify-content-between align-items-cent
er">
              <div>
                <div className="text-capitalize fw-semibold">{offer.provider}</div>
                <div className="fs-4 fw-bold">
                  {(offer.priceInPaise / 100).toLocaleString("en-IN", { style: "curre
ncy", currency: "INR", maximumFractionDigits: 0 })}
                  {index === firstValidIndex && <span className="badge text-bg-succes
s ms-2">Lowest price</span>}
                </div>
                <div className="small text-muted">{formatLastUpdated(offer)}</div>
                <div className="d-flex gap-2 mt-1">
                  <span className={`badge ${offer.availability === "in_stock" ? "text
-bg-success" : "text-bg-warning"}`}>
                    {offer.availability === "in_stock" ? "In stock" : offer.availabil
ity === "out_of_stock" ? "Out of stock" : "Unknown"}
                  </span>
                  {offer.isSample && <span className="badge bg-secondary-subtle text-
secondary">Sample data</span>}
                </div>
              </div>
              <div className="d-flex flex-column gap-2">
                <a href={offer.productUrl} target="_blank" rel="noopener noreferrer"
```

```
className="btn btn-outline-primary btn-sm">
                  View on {offer.provider}
                </a>
                <button className="btn btn-primary btn-sm" onClick={() => handleAddTo
Watchlist(offer.id)}>
                  Add to watchlist
                </button>
              </div>
            </div>
          </div>
        ))}
      </div>
      {/* Price-history chart is a Day 6/7 stretch goal per Document 1 Section 1.2 —
not implemented in the 7-day core scope. */}
    </div>
  );
}
```

H. Beginner explanation: `firstValidIndex` finds the first offer (in ascending price order) whose `lastStatus` isn't `"error"`, matching Document 1's rule that the "Lowest price" label goes on the cheapest *valid* offer, not necessarily the literal first row. The watchlist button redirects to login and (via a `returnTo` query param your Login page can read, same pattern as `ProtectedRoute`'s state) returns the user here afterward. **I. Environment configuration:** None new. **J. Testing:** Open a known fixture product's detail page while logged out — "Add to watchlist" should route to login; log in and confirm it successfully adds and shows a confirmation message. **K. Expected results:** Offers sorted ascending by price; exactly one "Lowest price" label on the cheapest valid offer. **L. Troubleshooting:** "Cannot read properties of undefined" from rendering before data loads — Section 14. **M. Acceptance criteria:** ACC-02 passes on this page specifically; page content that did load stays visible even if a later action (like add-to-watchlist) fails. **N. Git workflow:** `feature/product-search` (extends the same comparison feature) — see Section 15. **O. Completion checklist:** [ ] loading/empty/error states implemented [ ] sorted ascending [ ] Lowest price label text-based [ ] isSample badge shown. **P. AI-assisted debugging prompt:** "On ProductDetailPage, I get 'Cannot read properties of undefined (reading offers)' right after navigating here. Here's the component: [paste]. Find the smallest fix without changing the loading state logic."

FE-07 — Dashboard

A. Task ID and title: FE-07 — User dashboard (`/dashboard`). **B. Owner and reviewer:** Owner: Member 2. Reviewer: Member 1. Depends on FE-02, FE-03, and Member 3's watchlist/alerts/notifications read endpoints. **C. Purpose:** At-a-glance summary per Document 1 Section 13.7. **D. Prerequisites:** `getWatchlist`, `getAlerts`, `getNotifications` reachable. **E. Exact files:** Create `client/src/pages/DashboardPage.jsx`. **F. Commands:** None new. **G. Implementation:**

```
import { useState, useEffect } from "react";
  import { Link } from "react-router-dom";
  import * as api from "../services/api";

  export default function DashboardPage() {
    const [status, setStatus] = useState("loading");
    const [summary, setSummary] = useState(null);

    useEffect(() => {
      let cancelled = false;
      async function load() {
        setStatus("loading");
        try {
          const [watchlist, alerts, notifications] = await Promise.all([
            api.getWatchlist(),
            api.getAlerts(),
            api.getNotifications(),
          ]);
          if (cancelled) return;
          setSummary({
            watchlistCount: watchlist.items.length,
            activeAlertCount: alerts.alerts.filter((a) => a.status === "active").lengt
  h,
            recentNotifications: notifications.events.slice(0, 5),
            totalNotifications: notifications.events.length,
          });
          setStatus("success");
        } catch (err) {
          if (!cancelled) setStatus("error");
        }
      }
      load();
      return () => { cancelled = true; };
    }, []);

    if (status === "loading") {
      return (
        <div className="row g-3">
          {[1, 2, 3].map((i) => (
            <div className="col-md-4" key={i}>
              <div className="card placeholder-glow"><div className="card-body"><span c
  lassName="placeholder col-6" /></div></div>
```

```
          </div>
        ))}
      </div>
    );
  }

  if (status === "error") {
    return <div className="alert alert-danger">Something went wrong loading your dash
board. Please refresh.</div>;
  }

  if (summary.watchlistCount === 0 && summary.activeAlertCount === 0) {
    return (
      <div className="text-center py-5">
        <p className="mb-3">You&apos;re not tracking anything yet — search for a prod
uct to get started.</p>
        <Link to="/search" className="btn btn-primary">Search products</Link>
      </div>
    );
  }

  return (
    <div>
      <div className="row g-3 mb-4">
        <div className="col-md-4">
          <div className="card text-center"><div className="card-body"><div className
="fs-3 fw-bold">{summary.watchlistCount}</div><div className="text-muted small">Watch
list items</div></div></div>
        </div>
        <div className="col-md-4">
          <div className="card text-center"><div className="card-body"><div className
="fs-3 fw-bold">{summary.activeAlertCount}</div><div className="text-muted small">Act
ive alerts</div></div></div>
        </div>
        <div className="col-md-4">
          <div className="card text-center"><div className="card-body"><div className
="fs-3 fw-bold">{summary.totalNotifications}</div><div className="text-muted small">N
otifications</div></div></div>
        </div>
      </div>

      <h2 className="h6">Recent notifications</h2>
      {summary.recentNotifications.length === 0 ? (
```

```
          <p className="text-muted">No notifications yet.</p>
        ) : (
          <ul className="list-group mb-4">
            {summary.recentNotifications.map((event) => (
              <li key={event.id} className="list-group-item small">
                {new Date(event.createdAt).toLocaleString()} — {event.channel} — {even
t.status}
              </li>
            ))}
          </ul>
        )}

        <div className="d-flex gap-2 flex-wrap">
          <Link to="/watchlist" className="btn btn-outline-primary btn-sm">Watchlist</L
ink>
          <Link to="/alerts" className="btn btn-outline-primary btn-sm">Alerts</Link>
          <Link to="/settings/notifications" className="btn btn-outline-primary btn-s
m">Notification preferences</Link>
        </div>
      </div>
    );
}
```

H. Beginner explanation: `Promise.all` fires all three GET requests in parallel instead of one after another, so the dashboard loads in roughly the time of the slowest single call, not the sum of all three. The `cancelled` flag avoids a React warning ("state update on an unmounted component") if the user navigates away before the requests finish. **I. Environment configuration:** None new. **J. Testing:** As a user with no watchlist/alerts yet, confirm the empty state shows. Add a watchlist item and an alert, reload, confirm the tiles update. **K. Expected results:** Tile counts match what `/api/watchlist`, `/api/alerts`, `/api/notifications` actually return for that user. **L. Troubleshooting:** Section 14. **M. Acceptance criteria:** Empty state exact wording matches Document 1 Section 13.7; skeleton shown while loading. **N. Git workflow:** `feature/dashboard-watchlist` — see Section 15. **O. Completion checklist:** [ ] 3 tiles correct [ ] 5 most recent notifications shown [ ] quick links work [ ] empty state matches spec wording. **P. AI-assisted debugging prompt:** "My dashboard tiles always show 0 even though I have watchlist items. Here's DashboardPage.jsx and the Network response for /api/watchlist: [paste]. Find the smallest fix."

FE-08 — Watchlist page

A. Task ID and title: FE-08 — Watchlist page (`/watchlist`). **B. Owner and reviewer:** Owner: Member 2. Reviewer: Member 1. Depends on FE-02, FE-03. **C. Purpose:** Let the user view and manage their saved offers, per Document 1 Section 13.8. **D. Prerequisites:** `getWatchlist`, `removeFromWatchlist` reachable. **E. Exact files:** Create `client/src/pages/WatchlistPage.jsx`. **F. Commands:** None new. **G. Implementation:**

```
import { useState, useEffect, useCallback } from "react";
  import { Link, useNavigate } from "react-router-dom";
  import * as api from "../services/api";

  export default function WatchlistPage() {
    const [items, setItems] = useState([]);
    const [status, setStatus] = useState("loading");
    const navigate = useNavigate();

    const load = useCallback(async () => {
      setStatus("loading");
      try {
        const data = await api.getWatchlist();
        setItems(data.items);
        setStatus(data.items.length === 0 ? "empty" : "success");
      } catch (err) {
        setStatus("error");
      }
    }, []);

    useEffect(() => { load(); }, [load]);

    async function handleRemove(watchlistItemId) {
      const previous = items;
      setItems((cur) => cur.filter((i) => i.id !== watchlistItemId));
      try {
        await api.removeFromWatchlist(watchlistItemId);
      } catch (err) {
        setItems(previous); // roll back on failure
      }
    }

    if (status === "loading") return <div className="spinner-border text-primary" role
  ="status" />;
    if (status === "error") {
      return (
        <div className="alert alert-danger d-flex justify-content-between align-items-c
  enter">
          <span>Something went wrong loading your watchlist.</span>
          <button className="btn btn-outline-danger btn-sm" onClick={load}>Retry</butto
  n>
        </div>
```

```
    );
  }
  if (status === "empty") return <p className="text-muted">Nothing on your watchlist yet.</p>;

  return (
    <div className="d-flex flex-column gap-2">
      {items.map((item) => (
        <div key={item.id} className="card">
          <div className="card-body d-flex justify-content-between align-items-center">
            <div>
              <div className="fw-semibold">{item.offer.title}</div>
              <div className="small text-muted text-capitalize">{item.offer.provider}</div>
              <div className="fw-bold">
                {(item.offer.priceInPaise / 100).toLocaleString("en-IN", { style: "currency", currency: "INR", maximumFractionDigits: 0 })}
              </div>
              {item.offer.isSample && <span className="badge bg-secondary-subtle text-secondary">Sample data</span>}
            </div>
            <div className="d-flex gap-2">
              <button
                className="btn btn-primary btn-sm"
                onClick={() => navigate(`/alerts/new?offerId=${item.offer.id}`)}
              >
                Add alert
              </button>
              <button className="btn btn-outline-danger btn-sm" onClick={() => handleRemove(item.id)}>
                Remove
              </button>
            </div>
          </div>
        </div>
      ))}
    </div>
  );
}
```

H. Beginner explanation: The "Remove" button updates the UI immediately (removes the row from state) before the server confirms, then rolls back only if the request fails — this is called an **optimistic update**, and it makes the UI feel instant. "Add alert" pre-fills the offer by passing `offerId` as a query parameter, matching Document 1 Section 13.8 ("pre-fills the offerId on the Create Alert form"). **I. Environment configuration:** None new. **J. Testing:** Add an item via the product detail page, confirm it appears here; remove it, confirm it disappears and stays removed after a refresh; click "Add alert" and confirm the alert form opens pre-filled. **K. Expected results:** List matches the user's actual watchlist; removal persists. **L. Troubleshooting:** Section 14. **M. Acceptance criteria:** Empty state wording matches spec exactly ("Nothing on your watchlist yet"); remove is confirmed against the real backend, not just local state. **N. Git workflow:** `feature/dashboard-watchlist` — see Section 15. **O. Completion checklist:** [ ] list renders real data [ ] remove works and persists [ ] add-alert pre-fill works [ ] empty/loading/error states present. **P. AI-assisted debugging prompt:** "Clicking Remove on WatchlistPage removes the row visually but it reappears after I refresh the page. Here's WatchlistPage.jsx: [paste]. Find the smallest fix."

FE-09 — Create/edit alert form

A. Task ID and title: FE-09 — Alert create/edit form (`/alerts/new`, `/alerts/:alertId/edit`). **B. Owner and reviewer:** Owner: Member 2. Reviewer: Member 1 (coordinate with Member 3 on the Alert model fields per Document 1 Section 14's "Alert model fields" note). Depends on FE-02, FE-03, FE-08 (offerId query param). **C. Purpose:** Let a user define an alert condition, matching the exact alert logic in Document 1 Section 9. **D. Prerequisites:** `createAlert`, `updateAlert`, `getNotificationPreferences` (to check `telegramLinked`) reachable. **E. Exact files:** Create `client/src/pages/AlertFormPage.jsx`. **F. Commands:** None new. **G. Implementation:** Per Document 1 Section 13.9: radio for alert type, conditionally shown numeric field with ₹ prefix, channel checkboxes with Telegram disabled until linked, validation (exactly one of target/drop required by type, positive number, ≥1 channel).

```
import { useState, useEffect } from "react";
  import { useNavigate, useParams, useSearchParams } from "react-router-dom";
  import * as api from "../services/api";

  export default function AlertFormPage() {
    const { alertId } = useParams();
    const [searchParams] = useSearchParams();
    const offerIdFromQuery = searchParams.get("offerId");
    const navigate = useNavigate();
    const isEditMode = Boolean(alertId);

    const [offerSummary, setOfferSummary] = useState(null);
    const [type, setType] = useState("target_price");
    const [amountRupees, setAmountRupees] = useState("");
    const [channels, setChannels] = useState([]);
    const [telegramLinked, setTelegramLinked] = useState(false);
    const [status, setStatus] = useState("loading");
    const [errors, setErrors] = useState({});
    const [submitting, setSubmitting] = useState(false);
    const [serverError, setServerError] = useState("");

    useEffect(() => {
      async function load() {
        setStatus("loading");
        try {
          const prefs = await api.getNotificationPreferences();
          setTelegramLinked(prefs.telegramLinked);

          if (isEditMode) {
            const { alerts } = await api.getAlerts();
            const existing = alerts.find((a) => a.id === alertId);
            if (!existing) { setStatus("error"); return; }
            setType(existing.type);
            setAmountRupees(
              String((existing.type === "target_price" ? existing.targetPriceInPaise :
  existing.dropAmountInPaise) / 100)
            );
            setChannels(existing.channels);
            setOfferSummary({ id: existing.offerId });
          } else if (offerIdFromQuery) {
            setOfferSummary({ id: offerIdFromQuery });
          }
```

```
        setStatus("ready");
      } catch (err) {
        setStatus("error");
      }
    }
    load();
  }, [alertId, isEditMode, offerIdFromQuery]);

  function validate() {
    const e = {};
    const amount = Number(amountRupees);
    if (!amountRupees || Number.isNaN(amount) || amount <= 0) {
      e.amount = "Enter a positive number.";
    }
    if (channels.length === 0) {
      e.channels = "Select at least one notification channel.";
    }
    return e;
  }

  const liveErrors = validate();
  const isValid = Object.keys(liveErrors).length === 0;

  function toggleChannel(channel) {
    setChannels((cur) => (cur.includes(channel) ? cur.filter((c) => c !== channel) :
[...cur, channel]));
  }

  async function handleSubmit(e) {
    e.preventDefault();
    const v = validate();
    setErrors(v);
    if (Object.keys(v).length > 0) return;

    const amountInPaise = Math.round(Number(amountRupees) * 100);
    const payload = {
      offerId: offerSummary.id,
      type,
      channels,
      ...(type === "target_price"
        ? { targetPriceInPaise: amountInPaise }
        : { dropAmountInPaise: amountInPaise }),
    };
```

```
    setSubmitting(true);
    setServerError("");
    try {
      if (isEditMode) {
        await api.updateAlert(alertId, payload);
      } else {
        await api.createAlert(payload);
      }
      navigate("/alerts", { state: { toast: isEditMode ? "Alert updated" : "Alert cre
ated" } });
    } catch (err) {
      setServerError(err?.response?.data?.error?.message || "Could not save this aler
t — try again.");
    } finally {
      setSubmitting(false);
    }
  }

  if (status === "loading") return <div className="spinner-border text-primary" role
="status" />;
  if (status === "error") return <div className="alert alert-danger">Could not load t
his alert. Go back and try again.</div>;

  return (
    <div className="mx-auto" style={{ maxWidth: 480 }}>
      <h1 className="h4 mb-3">{isEditMode ? "Edit alert" : "Create alert"}</h1>
      {offerSummary && <p className="text-muted small">For offer: {offerSummary.id}</
p>}
      {serverError && <div className="alert alert-danger">{serverError}</div>}

      <form onSubmit={handleSubmit} noValidate>
        <div className="mb-3">
          <div className="form-check">
            <input
              type="radio" id="type-target" className="form-check-input"
              checked={type === "target_price"} onChange={() => setType("target_pric
e")}
            />
            <label htmlFor="type-target" className="form-check-label">Notify at or be
low a price</label>
          </div>
          <div className="form-check">
```

```
            <input
              type="radio" id="type-drop" className="form-check-input"
              checked={type === "fixed_drop"} onChange={() => setType("fixed_drop")}
            />
            <label htmlFor="type-drop" className="form-check-label">Notify on a price
drop of at least</label>
          </div>
        </div>

        <div className="mb-3">
          <label htmlFor="amount" className="form-label">
            {type === "target_price" ? "Target price (₹)" : "Drop amount (₹)"}
          </label>
          <div className="input-group">
            <span className="input-group-text">₹</span>
            <input
              id="amount" type="number" min="1" step="1"
              className={`form-control${errors.amount ? " is-invalid" : ""}`}
              value={amountRupees} onChange={(e) => setAmountRupees(e.target.value)}
            />
            {errors.amount && <div className="invalid-feedback">{errors.amount}</div
>}
          </div>
        </div>

        <fieldset className="mb-3">
          <legend className="col-form-label">Notification channels</legend>
          <div className="form-check">
            <input
              type="checkbox" id="chan-email" className="form-check-input"
              checked={channels.includes("email")} onChange={() => toggleChannel("ema
il")}
            />
            <label htmlFor="chan-email" className="form-check-label">Email</label>
          </div>
          <div className="form-check">
            <input
              type="checkbox" id="chan-telegram" className="form-check-input"
              checked={channels.includes("telegram")} disabled={!telegramLinked}
              onChange={() => toggleChannel("telegram")}
              title={!telegramLinked ? "Link your Telegram first" : undefined}
            />
            <label htmlFor="chan-telegram" className="form-check-label">
```

```
                Telegram {!telegramLinked && (
                  <small className="text-muted">
                    (<a href="/settings/notifications">link your Telegram first</a>)
                  </small>
                )}
              </label>
            </div>
            {errors.channels && <div className="text-danger small mt-1">{errors.channel
s}</div>}
          </fieldset>

          <button type="submit" className="btn btn-primary w-100" disabled={!isValid ||
submitting}>
            {submitting ? <span className="spinner-border spinner-border-sm me-2" /> :
null}
            {isEditMode ? "Save changes" : "Create alert"}
          </button>
        </form>
      </div>
    );
}
```

H. Beginner explanation: The form converts rupees to paise (`Math.round(Number(amountRupees) * 100)`) right before sending, because Document 1 Section 5's precision policy requires every price field to be an integer number of paise — never a float. Doing the conversion once, right at the submit boundary, keeps the rest of the component working in friendlier rupee units. **I. Environment configuration:** None new. **J. Testing:** Create a target-price alert on a watchlisted offer, confirm it appears correctly on the Alert management page with the right ₹ value. Try submitting with no channel selected — submit button should already be disabled. Try with Telegram unlinked — checkbox should be disabled with the "link your Telegram first" hint. **K. Expected results:** `POST /api/alerts` body matches exactly `{offerId, type, targetPriceInPaise | dropAmountInPaise, channels}`. **L. Troubleshooting:** Section 14. **M. Acceptance criteria:** Exactly one of `targetPriceInPaise` / `dropAmountInPaise` is ever sent, matching `type`; paise conversion has no floating-point rounding bug (test with values like ₹1499.99 → should become `149999`, not `149998.99999...`). **N. Git workflow:** `feature/alerts` — see Section 15. **O. Completion checklist:** [ ] both alert types work [ ] Telegram gating works [ ] edit mode pre-fills correctly [ ] paise conversion verified with a decimal rupee value. **P. AI-assisted debugging prompt:** "My target-price alert is being created with targetPriceInPaise off by one paisa for some rupee values, e.g. 1499.99. Here's the amount-to-paise conversion code: [paste]. Explain the floating-point issue and give the smallest correct fix."

FE-10 — Alert management page

A. Task ID and title: FE-10 — Alert management page (`/alerts`). **B. Owner and reviewer:** Owner: Member 2. Reviewer: Member 1. Depends on FE-02, FE-03, FE-09. **C. Purpose:** List, pause/resume, edit, and delete alerts, per Document 1 Section 13.10. **D. Prerequisites:** `getAlerts`, `updateAlert`, `deleteAlert` reachable. **E. Exact files:** Create `client/src/pages/AlertManagementPage.jsx`. **F. Commands:** None new. **G. Implementation:**

```
import { useState, useEffect, useCallback } from "react";
  import { Link, useLocation, useNavigate } from "react-router-dom";
  import * as api from "../services/api";

  function describeCondition(alert) {
    if (alert.type === "target_price") {
      return `Alert when ≤ ₹${(alert.targetPriceInPaise / 100).toLocaleString("en-IN")}
  `;
    }
    return `Alert on a ₹${(alert.dropAmountInPaise / 100).toLocaleString("en-IN")}+ dro
  p`;
  }

  export default function AlertManagementPage() {
    const [alerts, setAlerts] = useState([]);
    const [status, setStatus] = useState("loading");
    const [confirmDeleteId, setConfirmDeleteId] = useState(null);
    const location = useLocation();
    const navigate = useNavigate();
    const toast = location.state?.toast;

    const load = useCallback(async () => {
      setStatus("loading");
      try {
        const data = await api.getAlerts();
        setAlerts(data.alerts);
        setStatus(data.alerts.length === 0 ? "empty" : "success");
      } catch (err) {
        setStatus("error");
      }
    }, []);

    useEffect(() => { load(); }, [load]);

    async function togglePause(alert) {
      const nextStatus = alert.status === "active" ? "paused" : "active";
      setAlerts((cur) => cur.map((a) => (a.id === alert.id ? { ...a, status: nextStatus
  } : a)));
      try {
        await api.updateAlert(alert.id, { status: nextStatus });
      } catch (err) {
        load(); // reload the real state on failure
```

```
    }
  }

  async function handleDelete(alertId) {
    try {
      await api.deleteAlert(alertId);
      setAlerts((cur) => cur.filter((a) => a.id !== alertId));
      setConfirmDeleteId(null);
    } catch (err) {
      setConfirmDeleteId(null);
      load();
    }
  }

  if (status === "loading") return <div className="spinner-border text-primary" role
="status" />;
  if (status === "error") {
    return (
      <div className="alert alert-danger d-flex justify-content-between align-items-c
enter">
        <span>Something went wrong loading your alerts.</span>
        <button className="btn btn-outline-danger btn-sm" onClick={load}>Retry</butto
n>
      </div>
    );
  }
  if (status === "empty") {
    return <p className="text-muted">No alerts yet — add one from an offer&amp;apos;s pag
e.</p>;
  }

  return (
    <div>
      {toast && <div className="alert alert-success">{toast}</div>}
      <table className="table align-middle">
        <thead>
          <tr>
            <th>Condition</th>
            <th>Channels</th>
            <th>Status</th>
            <th>Last triggered</th>
            <th></th>
          </tr>
```

```
        </thead>
        <tbody>
          {alerts.map((alert) => (
            <tr key={alert.id}>
              <td>{describeCondition(alert)}</td>
              <td className="text-capitalize">{alert.channels.join(", ")}</td>
              <td>
                <span className={`badge ${alert.status === "active" ? "text-bg-succes
s" : "text-bg-secondary"}`}>
                  {alert.status}
                </span>
              </td>
              <td>{alert.lastTriggeredAt ? new Date(alert.lastTriggeredAt).toLocaleDa
teString() : "—"}</td>
              <td className="d-flex gap-2">
                <button className="btn btn-outline-primary btn-sm" onClick={() => nav
igate(`/alerts/${alert.id}/edit`)}>
                  Edit
                </button>
                <button className="btn btn-outline-secondary btn-sm" onClick={() => t
ogglePause(alert)}>
                  {alert.status === "active" ? "Pause" : "Resume"}
                </button>
                {confirmDeleteId === alert.id ? (
                  <>
                    <button className="btn btn-danger btn-sm" onClick={() => handleDe
lete(alert.id)}>Confirm</button>
                    <button className="btn btn-outline-secondary btn-sm" onClick={()
=> setConfirmDeleteId(null)}>Cancel</button>
                  </>
                ) : (
                  <button className="btn btn-outline-danger btn-sm" onClick={() => se
tConfirmDeleteId(alert.id)}>Delete</button>
                )}
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

H. Beginner explanation: `describeCondition` turns the raw stored fields (`targetPriceInPaise` / `dropAmountInPaise`) into the plain-language sentence Document 1 Section 13.10 specifies ("Alert when ≤ ₹14,999"). The delete button uses a simple two-click confirm pattern (click once to arm it, click "Confirm" to actually delete) rather than a browser `confirm()` dialog, which is easier to style and test. **I. Environment configuration:** None new. **J. Testing:** Create two alerts via FE-09, confirm both list correctly with the right plain-language condition; pause one, confirm the badge updates and persists on reload; delete one with the confirm flow. **K. Expected results:** List matches server state exactly after each action. **L. Troubleshooting:** Section 14. **M. Acceptance criteria:** Plain-language condition text matches the Document 1 wording pattern; delete requires an explicit confirm step. **N. Git workflow:** `feature/alerts` — see Section 15. **O. Completion checklist:** [ ] list/pause/resume/edit/delete all work against the real backend [ ] empty state matches spec wording [ ] delete has a confirm step. **P. AI-assisted debugging prompt:** "Pausing an alert on AlertManagementPage shows 'active' again after I refresh the page even though I clicked Pause. Here's togglePause and the component: [paste]. Find the smallest fix."

FE-11 — Notification preferences, Telegram linking, and notification history

A. Task ID and title: FE-11 — Notification preferences page (`/settings/notifications`) and notification history page (`/notifications`). **B. Owner and reviewer:** Owner: Member 2. Reviewer: Member 1. Depends on FE-02, FE-03, and Member 4's Telegram linking flow (Document 1 Section 6.5) being reachable. **C. Purpose:** Let the user see their notification email, link/unlink Telegram, and review their notification history, per Document 1 Sections 13.11–13.12. **D. Prerequisites:** `getNotificationPreferences`, `updateNotificationPreferences`, `linkTelegram`, `getNotifications` reachable. **E. Exact files:** Create `client/src/pages/NotificationPreferencesPage.jsx`, `client/src/pages/NotificationHistoryPage.jsx`. **F. Commands:** None new. **G. Implementation:**

`client/src/pages/NotificationPreferencesPage.jsx` (Document 1 Section 13.11 — email read-only, Telegram link/unlink flow with a code the user sends via `/start <code>`; the exact code-generation mechanism is owned by Member 4 per Document 1 Section 6.5, so this page calls `api.linkTelegram(linkCode)` with whatever code the user types in, matching the documented endpoint contract exactly):

```
import { useState, useEffect } from "react";
  import * as api from "../services/api";

  export default function NotificationPreferencesPage() {
    const [prefs, setPrefs] = useState(null);
    const [status, setStatus] = useState("loading");
    const [linkCode, setLinkCode] = useState("");
    const [linkStatus, setLinkStatus] = useState("idle"); // idle | submitting | succes
  s | error
    const [linkError, setLinkError] = useState("");

    async function load() {
      setStatus("loading");
      try {
        const data = await api.getNotificationPreferences();
        setPrefs(data);
        setStatus("success");
      } catch (err) {
        setStatus("error");
      }
    }

    useEffect(() => { load(); }, []);

    async function handleLinkSubmit(e) {
      e.preventDefault();
      setLinkStatus("submitting");
      setLinkError("");
      try {
        await api.linkTelegram(linkCode);
        setLinkStatus("success");
        await load(); // refresh telegramLinked
      } catch (err) {
        setLinkStatus("error");
        setLinkError(err?.response?.data?.error?.message || "Code expired, generate a n
  ew one.");
      }
    }

    if (status === "loading") return <div className="spinner-border text-primary" role
  ="status" />;
    if (status === "error") return <div className="alert alert-danger">Could not load n
```

```
otification preferences. Refresh to try again.</div>;

  return (
    <div className="mx-auto" style={{ maxWidth: 480 }}>
      <h1 className="h4 mb-3">Notification preferences</h1>

      <div className="mb-4">
        <label className="form-label">Email</label>
        <input type="email" className="form-control" value={prefs.email} readOnly dis
abled />
        <div className="form-text">Notifications go to your account email for this ve
rsion of PriceRadar.</div>
      </div>

      <div className="mb-4">
        <label className="form-label d-block">Telegram</label>
        {prefs.telegramLinked ? (
          <span className="badge text-bg-success">Telegram linked ✓</span>
        ) : (
          <>
            <p className="small text-muted">
              Open the PriceRadar Telegram bot, send <code>/start</code>, then paste
the code it gives you below.
            </p>
            {linkStatus === "success" ? (
              <div className="alert alert-success">Telegram linked successfully.</div
>
            ) : (
              <form onSubmit={handleLinkSubmit} className="d-flex gap-2">
                <input
                  type="text" className="form-control" placeholder="Paste your link c
ode"
                  value={linkCode} onChange={(e) => setLinkCode(e.target.value)}
                />
                <button type="submit" className="btn btn-primary" disabled={!linkCode
|| linkStatus === "submitting"}>
                  {linkStatus === "submitting" ? "Linking…" : "Link Telegram"}
                </button>
              </form>
            )}
            {linkStatus === "error" && <div className="text-danger small mt-2">{linkE
rror}</div>}
          </>
```

```
          )}
        </div>
      </div>
    );
  }
```

`client/src/pages/NotificationHistoryPage.jsx` (Document 1 Section 13.12):

```
import { useState, useEffect } from "react";
  import * as api from "../services/api";

  const STATUS_LABELS = {
    sent: "Sent",
    failed: "Failed",
    skipped_duplicate: "Skipped — duplicate",
  };

  const STATUS_BADGES = {
    sent: "text-bg-success",
    failed: "text-bg-danger",
    skipped_duplicate: "text-bg-secondary",
  };

  export default function NotificationHistoryPage() {
    const [events, setEvents] = useState([]);
    const [status, setStatus] = useState("loading");
    const [expandedId, setExpandedId] = useState(null);

    useEffect(() => {
      async function load() {
        setStatus("loading");
        try {
          const data = await api.getNotifications();
          setEvents(data.events);
          setStatus(data.events.length === 0 ? "empty" : "success");
        } catch (err) {
          setStatus("error");
        }
      }
      load();
    }, []);

    if (status === "loading") return <div className="spinner-border text-primary" role
  ="status" />;
    if (status === "error") return <div className="alert alert-danger">Something went w
  rong loading notification history.</div>;
    if (status === "empty") return <p className="text-muted">No notifications yet.</p>;

    return (
      <table className="table align-middle">
```

```
        <thead>
          <tr>
            <th>Date</th>
            <th>Channel</th>
            <th>Status</th>
            <th>Message</th>
          </tr>
        </thead>
        <tbody>
          {events.map((event) => (
            <tr key={event.id}>
              <td className="small">{new Date(event.createdAt).toLocaleString()}</td>
              <td className="text-capitalize">{event.channel}</td>
              <td><span className={`badge ${STATUS_BADGES[event.status]}`}>{STATUS_LABE
LS[event.status]}</span></td>
              <td className="small">
                {expandedId === event.id || event.message.length <= 80
                  ? event.message
                  : `${event.message.slice(0, 80)}…`}
                {event.message.length > 80 && (
                  <button
                    className="btn btn-link btn-sm p-0 ms-1"
                    onClick={() => setExpandedId(expandedId === event.id ? null : even
t.id)}
                  >
                    {expandedId === event.id ? "Show less" : "Show more"}
                  </button>
                )}
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    );
}
```

H. Beginner explanation: The Telegram linking flow calls exactly `POST /api/telegram/link` with `{linkCode}` as Document 1 Section 6.5 specifies — the frontend never talks to Telegram directly, and never needs the bot token (that stays server-side with Member 4, per Document 1 Section 7.1's secrecy rules). This page just submits whatever code the user pastes in and reports success or failure. **I. Environment configuration:** None new. **J. Testing:** With Telegram unlinked, confirm the link form shows; submit a bad code and confirm "Code expired, generate a new one" (or the server's actual message) appears; once Member 4's backend linking works end-to-end, confirm a real `/start <code>`

flow results in `telegramLinked: true` on reload. **K. Expected results:** `telegramLinked` flips to `true` after a successful link; notification history table matches `/api/notifications` exactly. **L. Troubleshooting:** Section 14. **M. Acceptance criteria:** Telegram checkbox on the alert form (FE-09) correctly reflects `telegramLinked` after this page links it; notification history empty/loading/error states match spec wording. **N. Git workflow:** `feature/notification-settings` — see Section 15. **O. Completion checklist:** [ ] email shown read-only [ ] Telegram link/unlink flow works against the real backend [ ] notification history renders with correct status labels [ ] long messages truncate with show more/less. **P. AI-assisted debugging prompt:** "After a successful Telegram link, the alert form's Telegram checkbox is still disabled. Here's AlertFormPage.jsx's telegramLinked logic and NotificationPreferencesPage.jsx: [paste]. Find the smallest fix — likely a stale read of preferences."

---

10. Loading / empty / error / stale / success state pattern

Every page above follows the same small state machine rather than inventing its own per page — this consistency is what keeps a 4-person, 7-day project maintainable (Document 1 Section 1.3: "no feature is implemented in a way that only one team member understands"). The convention used throughout this guide:

```
const [status, setStatus] = useState("loading"); // loading | empty | success | error
```

- **loading** — show a Bootstrap spinner (`<div className="spinner-border" role="status">`) or, for list/table pages, a skeleton (`placeholder-glow`).
- **empty** — show the exact wording Document 1 Section 13 specifies for that page (never a generic "No data").
- **error** — show a message plus, where the action is retryable, a "Retry" button that re-runs the same load function. Never let an error state remove content that was already successfully loaded elsewhere on the page (Document 1 Section 13.4: "page content that did load stays visible").
- **success** — render the real data.
- **stale** (offer-specific, not a page-level status) — handled inline via `lastStatus` / `lastSuccessAt` formatting, as shown in FE-06's `formatLastUpdated()`.

If you want a reusable wrapper instead of repeating the four-branch `if` in every page, this optional component captures the same convention:

```
// client/src/components/StatusWrapper.jsx
  export default function StatusWrapper({ status, loading, empty, error, onRetry, child
  ren }) {
    if (status === "loading") return loading;
    if (status === "empty") return empty;
    if (status === "error") {
      return (
        <div className="alert alert-danger d-flex justify-content-between align-items-c
  enter">
          <span>{error}</span>
          {onRetry && <button className="btn btn-outline-danger btn-sm" onClick={onRetr
  y}>Retry</button>}
        </div>
      );
    }
    return children;
  }
```

Using it is optional — the explicit `if` branches shown in every FE-0x task are equally correct and arguably easier for a beginner to read and debug. Pick one convention and use it consistently across all your pages; don't mix styles page-to-page.

---

11. Form validation pattern

Every form in this project (Register, Login, Create/Edit Alert) follows the same three rules:

1. **Validate on every render**, not just on submit — compute a `validate()` function that returns an errors object, and derive `isValid` from whether that object is empty. This means the submit button's `disabled` state is always in sync with the form, with no separate "has the user touched this field" bookkeeping needed for a 7-day MVP.
2. **Show inline error text under the specific field**, not a single banner for the whole form — this is what lets a user immediately see *which* field is wrong (as shown in `FormField.jsx` in FE-05, with Bootstrap's `is-invalid` / `invalid-feedback` classes).
3. **Disable the submit button while invalid or while a request is pending** — never let a user click submit twice while a request is in flight, and never let them submit a form you already know is invalid client-side.

Worked example (condensed from FE-05's `RegisterPage.jsx`):

```
function validate(form) {
    const e = {};
    if (!form.email.trim()) e.email = "Email is required.";
    else if (!/^[^\s@]+@[^\s@]+.[^\s@]+$/.test(form.email)) e.email = "Enter a valid em
  ail address.";
    if (!form.password || form.password.length < 8) e.password = "Password must be at l
  east 8 characters.";
    return e;
  }

  // In the component:
  const liveErrors = validate(form);
  const isValid = Object.keys(liveErrors).length === 0;

  <button type="submit" disabled={!isValid || submitting}>Submit</button>
```

Client-side validation is a convenience, not a security boundary — the backend (Member 3's work, via `express-validator`, per Document 1 Section 8) re-validates everything independently. Never assume that because the frontend blocked a bad submission, the backend doesn't need to check it too.

---

12. Responsive styling

Bootstrap 5's grid and utility classes handle responsiveness without any custom media queries for most of this project:

- **Grid:** `row` / `col-md-4` etc. (as used in the Dashboard's three tiles in FE-07) automatically stack to full width below the `md` breakpoint (768px) and sit side-by-side above it — this satisfies Document 1 Section 13.1's "hero stacks vertically under 768px" requirement with zero extra CSS.
- **Spacing utilities:** `p-*`, `m-*`, `gap-*`, `g-*` map directly onto Document 1 Section 13's spacing scale (4/8/12/16/24/32px) — e.g. `gap-3` ≈ 16px, matching the "16–24px gaps between cards" instruction. Don't invent a new spacing scale in `theme.css`; reach for Bootstrap's utility classes first.
- **Cards:** every offer/product/alert row in this guide uses Bootstrap's `.card` / `.card-body`, matching "Bootstrap 5 cards for offer cards and dashboard tiles" (Document 1 Section 13).
- **Flex utilities:** `d-flex`, `flex-column`, `justify-content-between`, `align-items-center` are used throughout instead of custom flexbox CSS.

`client/src/styles/theme.css` should contain only what Bootstrap doesn't already provide — specifically, the project's exact palette from Document 1 Section 13, since Bootstrap's default blue/green don't match it:

```
:root {
    --pr-primary: #0F6E56;
    --pr-text: #1C1C1A;
    --pr-muted: #6B6B66;
    --pr-bg: #F5F5F4;
    --pr-surface: #FFFFFF;
    --pr-border: #E2E1DB;
    --pr-success: #1D9E75;
    --pr-warning: #B8860B;
    --pr-error: #A32D2D;
    --pr-sample-bg: #ECECE8;
    --pr-sample-text: #6B6B66;
  }

  body {
    background-color: var(--pr-bg);
    color: var(--pr-text);
  }

  .btn-primary {
    background-color: var(--pr-primary);
    border-color: var(--pr-primary);
  }

  a {
    color: var(--pr-primary);
  }
```

Do not introduce a different color anywhere in the app — every palette value is already decided in Document 1 Section 13's opening paragraph. If a page seems to need a color that isn't listed there, that's a sign to re-check the spec, not to pick one.

---

13. Testing

Document 1 Section 3 confirms there is no automated frontend test framework in this project's scope — Jest + Supertest covers the backend only. Member 2's testing responsibility is **manual verification against the real local backend**, not against imagined data, and must happen before Day 6 (Document 1 Section 15's Day 6 objective is stabilization, which assumes integration already works).

13.1 Before you start manual testing

```
# In one PowerShell window:
  cd D:\Projects\PriceRadar\server
  npm run dev

  # In a second PowerShell window:
  cd D:\Projects\PriceRadar\client
  npm run dev
```

Confirm `http://localhost:5000/api/health` returns `{"status":"ok", ...}` in a browser before testing any page — if the backend isn't running, every page will show its error state, which can look like a frontend bug when it isn't.

13.2 Manual test checklist, per page

Page	What to click	What you should see
Home (`/`)	Load the page	Hero, search bar, nav shows Login/Register when logged out
Search (`/search`)	Search a known fixture product name	Results with ≥2 offers, "Lowest price" label, "Sample data" badges where applicable
Search (`/search`)	Search gibberish like `"zzzqqq123"`	Empty state: "No products matched..."
Search (`/search`)	Stop the backend, search again	Error state with a working Retry button
Product detail	Click a result card	Offers sorted by price, correct "Lowest price" placement, correct stale/error timestamp text
Product detail	Click "Add to watchlist" while logged out	Redirected to Login
Register	Submit with mismatched passwords	Inline error under Confirm password, submit stays disabled
Register	Register with an already-used email	Inline error under Email ("already registered" or similar), not a generic banner
Login	Wrong password	"Email or password is incorrect"
Login	Arrive here via a ProtectedRoute redirect, then log in	Lands back on the original page, not always `/dashboard`
Dashboard	Load with no watchlist/alerts	Empty-state message and a Search button
Dashboard	Load with data	Correct tile counts matching the real `/api/watchlist`, `/api/alerts`, `/api/notifications`
Watchlist	Remove an item	Disappears immediately, stays gone after refresh
Watchlist	Click "Add alert"	Alert form opens pre-filled with that offer
Alert form	Try submitting with 0 channels selected	Submit stays disabled
Alert form	Create a target-price alert with a decimal rupee value (e.g. 1499.50)	`targetPriceInPaise` sent is exactly `149950`, no floating-point drift
Alert management	Pause, then refresh the page	Still shows paused
Alert management	Delete, cancel, then delete again and confirm	Row removed only after explicit confirm
Notification preferences	Submit an invalid/expired Telegram code	"Code expired, generate a new one" (or server's message), not a crash
Notification history	Load with no notifications yet	"No notifications yet."

Page	What to click	What you should see
Any protected page	Visit directly while logged out	Redirect to `/login` with "Please log in to continue."
Any route	Visit a nonsense URL	404 "Page not found" with a link home

13.3 Cross-user isolation check (ties to ACC-03)

Register two separate accounts. Log in as User A, add a watchlist item and an alert. Log out, log in as User B. Confirm User B's Watchlist and Alerts pages are empty — if User B ever sees User A's data, that's a backend bug (Member 3's `req.user.id` scoping, Document 1 Section 8), but it's Member 2's job to actually run this check and report it, since the frontend is what surfaces it.

---

14. Debugging

Common beginner frontend errors in this stack, and how to fix each:

14.1 CORS errors in the browser console

Symptom: console shows something like `Access to XMLHttpRequest at 'http://localhost:5000/api/...' from origin 'http://localhost:5173' has been blocked by CORS policy`.

What it means: the backend's CORS configuration (`app.use(cors({ origin: process.env.CLIENT_ORIGIN }))`, Document 1 Section 8) doesn't recognize your frontend's URL as an allowed origin.

Fix: check `server/.env`'s `CLIENT_ORIGIN` value — it must exactly match the URL shown by `npm run dev` in `client/` (typically `http://localhost:5173`), including the protocol and port, with no trailing slash. This is a `server/.env` value, so you can't fix it yourself — flag it to Member 3, but you can verify the mismatch yourself first by comparing the two values.

14.2 A blank page with no error visible

Symptom: navigating to a route shows nothing, but the browser console might not show an obvious error — check it anyway (F12 → Console tab).

Common cause: a typo in `App.jsx`'s route table — e.g. `<Route path="/producs/:productId" ...>` (missing a `t`) means `/products/123` matches nothing and renders your catch-all `*` route (or nothing, depending on route order), while the page you expected never mounts.

Fix: double check every `path` string in `App.jsx` against the table in Section 7.1 character-for-character, and confirm the catch-all `<Route path="*" ...>` is listed **last**.

14.3 "Cannot read properties of undefined (reading '...')"

Symptom: a crash the instant a page loads, referencing some property of data that hasn't arrived yet.

Cause: the component tries to render `data.product.title` (etc.) before the API call has resolved — i.e., rendering happens during the `"loading"` state as if it were already `"success"`.

Fix: make sure every page's JSX only accesses the loaded data inside the `status === "success"` branch (see Section 10's pattern) — never unconditionally at the top of the render, and never with `data.something` before checking `data` itself isn't `null`.

14.4 401 Unauthorized on a request that should be authenticated

Symptom: a protected endpoint call (e.g. `api.getWatchlist()`) returns 401 even though the user just logged in.

Likely causes, in order of likelihood:

1. The Axios interceptor in `api.js` (Section 5.3) isn't attached to the instance actually being used — double check you're importing from `services/api.js` and not accidentally creating a second `axios.create()` somewhere.
2. The localStorage key is mismatched — `api.js` and `AuthContext.jsx` must both use exactly `"pricradar_token"`.
3. The token expired (`JWT_EXPIRES_IN=7d` on the backend) — log out and back in.

Fix: open the browser's Network tab, click the failing request, check the Request Headers — confirm an `Authorization: Bearer ...` header is actually present. If it's missing, it's the interceptor or the storage key; if it's present but still 401, it's likely an expired or malformed token (check with Member 3).

14.5 Stale data showing after a mutation

Symptom: you add/remove/update something and the UI doesn't reflect it until a manual refresh.

Fix: after any `create` / `update` / `delete` API call, either re-fetch the list (call your `load()` function again) or update local state directly — don't assume React will somehow "know" the server changed. See the optimistic-update pattern in FE-08's `handleRemove`.

---

15. Git workflow

Use **exactly** this branch list — do not invent different names:

```
main
  feature/project-scaffold
  feature/product-search
  feature/user-authentication
  feature/dashboard-watchlist
  feature/alerts
  feature/notification-settings
  feature/retailer-provider
  feature/price-monitor
  feature/email-notifications
  feature/telegram-notifications
  feature/deployment
```

As Member 2, you'll mainly work on `feature/project-scaffold`, `feature/product-search`, `feature/user-authentication`, `feature/dashboard-watchlist`, `feature/alerts`, and `feature/notification-settings` — the others belong to Members 1/3/4, but the repo is shared, so never push directly to `main`.

Full Git mechanics — installing Git, configuring identity, resolving merge conflicts, reverting a bad commit, recovering from mistakes — are covered completely in **Document 5**. What follows is the one example workflow for `feature/product-search`; the exact same pattern applies to every other frontend branch above — just swap the branch name and commit message.

```
# 1. Start from an up-to-date main
  git switch main
  git pull --ff-only origin main

  # 2. Create your feature branch
  git switch -c feature/product-search

  # 3. Implement the feature (SearchPage.jsx, ProductCard.jsx, OfferRow.jsx — see Secti
  on 8)
  #    ... write code, save, test in the browser against the running backend ...

  # 4. Before staging anything, look at what actually changed
  git status
  git diff

  # 5. Stage only the files this feature touched — never `git add -A` blindly
  git add client/src/pages/SearchPage.jsx client/src/components/ProductCard.jsx client/
  src/components/OfferRow.jsx

  # 6. Double-check exactly what you're about to commit
  git diff --cached

  # 7. Commit with a conventional-commit message
  git commit -m "feat: implement product search and results page"

  # 8. Push the branch and open a PR
  git push -u origin feature/product-search
```

Then open the PR on GitHub (branch → `main`), describe what you built, and request a review from Member 1 (API contract/auth-sensitive areas) or any teammate for everything else, per Document 1 Section 14's review rule. Do not merge your own PR without a review.

A few reminders specific to frontend work:

- Run `git status` and `git diff` before every `git add` — it's easy to accidentally stage an unrelated file you were experimenting with (e.g. a stray `console.log` in `main.jsx`).
- Never commit `client/.env` — only `client/.env.example`. If `git status` shows `.env` as untracked-but-about-to-be-added, stop and check `client/.gitignore`.
- Never `git add -A` or `git add .` on this project — always add specific paths, as shown above, so you don't accidentally sweep in files from a half-finished feature.

---

16. Acceptance criteria

Member 2's work is considered done when all of the following hold, verified against the **real local backend running** (not imagined data), per Document 1's Definition of Done (Section 16):

Tied to Document 1 Section 17's frontend-relevant acceptance tests:

- [ ] **ACC-02** — On every page showing a comparison (Search results, Product detail), the lowest valid offer is marked with the literal text "Lowest price," never color alone.
- [ ] **ACC-03 (UI consequence)** — Logging in as a second user never shows the first user's watchlist, alerts, or notification history anywhere in the UI (verified manually per Section 13.3).
- [ ] **ACC-10** — Every offer with `isSample: true` shows a visible "Sample data" badge, on every page that renders an offer (Search results, Product detail, Watchlist).

Frontend-specific criteria:

- [ ] Every page listed in Document 1 Section 13 has working loading, empty, and error states, using the exact wording Section 13 specifies where it specifies wording.
- [ ] Every form (Register, Login, Create/Edit Alert) validates client-side before submission, shows inline per-field errors, and disables its submit button while invalid or while a request is in flight.
- [ ] No page ever renders a blank white screen on an API error — every error path shows the shared pattern from Section 10 (a message, and a Retry button where the action is retryable).
- [ ] `ProtectedRoute` correctly gates every page marked "Yes" in Section 7.1's table and returns the user to their original destination after login.
- [ ] No frontend file ever reads, logs, or hardcodes a secret — the only environment variable the frontend reads is `VITE_API_URL`.
- [ ] Every API call goes through `client/src/services/api.js` — no component calls `fetch()` or creates its own Axios instance.
- [ ] The app has been exercised end-to-end against the real backend (`npm run dev` in both `client/` and `server/`) at least once before Day 6, per Section 13.1 above.
- [ ] Every frontend feature branch has been merged to `main` via a reviewed pull request, per Section 15.